Chapter 2
File Organization and Storage

How rows become pages on disk, and why that decides performance

Advanced Data Management

Based on Chapter 2 slides

How to read this chapter

Chapter 1 drew the map; this chapter opens the bottom box, the storage manager, and specifically the part that decides how bytes live on disk. The through-line is a single claim: *everything a database does eventually reads or writes a page, so how we lay pages out decides how fast the whole system is.* By the end you should be able to look at a table, estimate how many pages it occupies, and predict how many of them a given operation will touch. That number, pages touched, is the currency of the rest of the course.

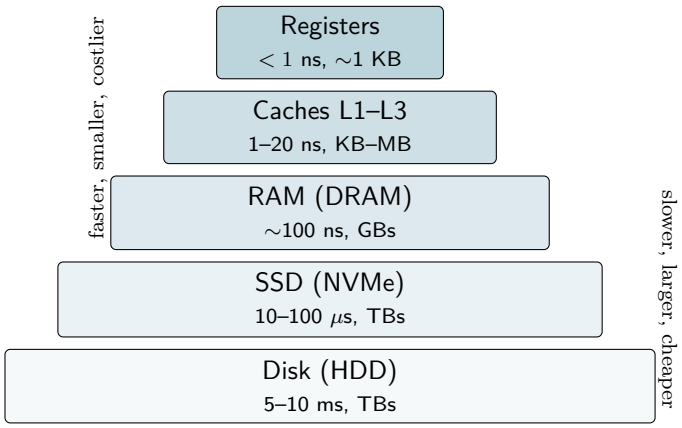
1 Refresher: The Memory Hierarchy and OS Paging

1.1 Motivation: why start with hardware?

A database's job, stripped to its essence, is to move data between slow, large storage and fast, small memory, efficiently. Every design choice in this chapter is really a way of avoiding slow reads. To reason about that, you need a feel for *how* slow each level is, because the differences are not small; they are astronomical.

Key insight. For a database, the number of disk pages read is a better cost measure than the number of CPU instructions. This section justifies that claim.

1.2 The hierarchy



The numbers are orders of magnitude, not exact; the gaps are the point. Each step down is much larger and much slower. The CPU can only compute on what is in registers; everything else has to be brought up the pyramid first.

Worked example: The gap, dramatised

Human intuition is bad at nanoseconds, so rescale. If one L1 cache access took *one second*, the rest would take:

Level	Real latency	If L1 = 1 second
L1 cache	∼1 ns	1 second
RAM	∼100 ns	about 1.5 minutes
SSD	∼100 µs	about 1 day
Disk (HDD)	∼10 ms	about 4 months

Imagine asking for a number and waiting four months for it. That is what a disk read looks like to the processor. A database lives or dies by how few disk pages it reads. This is also why a query that touches a few extra pages "by accident" can be ten times slower than one that does not, even though the CPU work is identical.

1.3 Sequential versus random access

Not all reads on the same device cost the same.

- **Sequential:** read blocks that sit next to each other. The device streams them at full bandwidth; on an HDD the head stays put and the platter just spins underneath.
- **Random:** jump around. On an HDD every jump is a physical seek of the arm, about 10 ms. A thousand random reads is ten seconds; a thousand sequential reads is a few milliseconds.
- **SSDs** have no moving parts, so random access is far less painful, but still slower than sequential, because reads happen in fixed-size blocks and every request carries overhead.

The design consequence runs through the whole chapter: databases prefer to read data in large sequential runs of pages. Much of storage layout is about turning "random" access patterns into "sequential" ones.

In the real world. A typical HDD does about 100–200 random reads per second but streams 100–200 MB/s sequentially. That is the difference between reading 1 MB in 5 ms and reading it in 5 seconds, if it were scattered in 4 KB pieces.

1.4 Blocks and pages: the unit of transfer

Concept: You pay per page, not per byte

- Disks and SSDs do not read one byte. They read a whole **block** (typically 4 KB).
- The OS and the DBMS group data into fixed-size **pages**. PostgreSQL uses 8 KB pages.
- To read one row, you read its entire page into memory. The other rows on that page come along for free, whether you wanted them or not.
- So the real cost question is never "how many bytes?" but **"how many distinct pages must I touch?"**

Because a page is the smallest thing we move, packing *related* rows into the *same* page directly reduces I/O. If the ten rows you need are on one page, that is one read; if they are spread over ten pages, that is ten reads for the same ten rows. This is why layout matters, and it is the idea behind every technique in this chapter.

1.5 OS paging and virtual memory, the quick tour

Your programs already rely on a paging system without you noticing.

- Each process sees *virtual* memory, split into fixed-size pages (often 4 KB).
- A *page table* maps virtual pages to physical RAM frames.
- If a page is not in RAM, a *page fault* pulls it in from disk, transparently but slowly.
- A *TLB* (translation lookaside buffer) caches recent translations so lookups stay fast.

A DBMS uses the same idea at its own level: it keeps a pool of pages in RAM (the **buffer pool**) and faults the rest in from disk. But it manages this *itself* rather than leaving it to the OS, and Section 7 explains why. For now, just note that the DBMS and the OS are doing the same kind of work, and that the DBMS does it on purpose.

Summary

Memory is a hierarchy; each level down is far larger and far slower. Disk is 10^4 to 10^5 times slower than RAM, so I/O dominates. Data moves in fixed-size pages; you pay per page. Sequential beats random, so locality is everything. With this in hand we can measure storage designs by the one number that matters: pages read.

2 The I/O Cost Model

2.1 Counting I/Os

Concept: The cost model used for the rest of the course

We estimate the cost of an operation as the **number of page reads and writes**.

- Let B be the number of pages a table occupies.
- A full scan reads all B pages: cost $= B$.
- CPU work per page is ignored, because it is usually dwarfed by the cost of fetching the page.

Why is this simplification fair? Reading a page from disk can take as long as *millions* of CPU cycles. Counting pages captures where the time really goes, and it is easy to reason about with pencil and paper. When we later say an index lookup costs $\log_2 B$ and a heap scan costs B, we mean pages, and the comparison is meaningful precisely because each page costs roughly the same.

The model has known blind spots, and you should know them too: it treats sequential and random reads as equal (they are not), it ignores whether a page is already cached (Chapter 6 fixes that), and it ignores CPU-heavy operations such as sorting in memory. It is a first approximation, and a remarkably useful one.

Warning / Gotcha: When CPU dominates, and how to notice

The page count misleads you whenever an operator does a lot of work per page. Three common cases: a sort or hash build over a table that fits in memory, where the one pass of reads is cheap but comparing or hashing millions of rows is not; a filter with an expensive function per row, such as a regular expression or a `lower()` on a long string over every row of a scan; and an aggregate with many groups, where every row is a hash-table probe. In each case the I/O model says "B pages, cheap" and the clock disagrees.

You notice it in `EXPLAIN (ANALYZE, BUFFERS)`: the node shows a *small* number of buffers (all `shared hit`, no `read`) yet a *large* actual time, and that time stays the same on a second, fully cached run. Pure I/O nodes get much faster once cached; CPU-bound nodes do not. A `Sort` node with `Sort Method: quicksort Memory: ...` that still takes seconds is the textbook example: zero disk, all CPU.

2.2 A first cost intuition

Trade-offs: Cheap versus expensive

Cheap	Expensive
Reading pages you already have in memory.	Random single-page reads from disk.
Reading many pages sequentially.	Scanning a whole table to find one row.
Touching few, well-packed pages.	Spreading related rows across many pages.

Indexes (Chapter 3) and the buffer pool (Chapter 6) exist to push work from the right column to the left.

Worked example: Pause and check: counting I/Os on a heap file

A heap file for `orders` occupies $B = 10{,}000$ pages. There is no index.
1. How many page reads to scan the whole table?
2. Worst case to find the single row with `id = 4275`?
3. Average case, if the row exists and is equally likely to be anywhere?

1: 10,000. A full scan touches every page. 2: 10,000. In the worst case the row is on the last page; and if the row is *absent*, we must read every page to be sure, so absence always costs the full B. 3: about 5,000, that is $B/2$. On average we find it halfway through and can stop, because `id` is unique so there is nothing more to find.
At 10 ms per random read on an HDD, case 3 is 50 seconds to find one row. This is precisely the pain an index removes: from $O(B)$ to $O(\log B)$ or $O(1)$, which for this table is 14 reads or 1 read instead of 5,000.

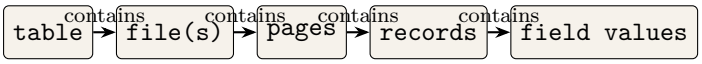
Summary

Cost = pages touched. A table of B pages costs B to scan, B to search for an absent key, and $B/2$ on average to find a present unique key. Everything that follows is judged by this one number.

3 Pages and Records

3.1 From table to file to page to record

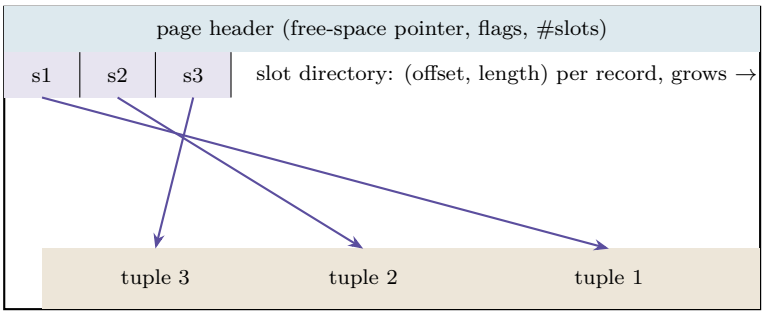
Four levels of containment, each inside the previous:



A table is stored as one or more files on disk. A file is a sequence of fixed-size pages. A page holds many records (rows) plus some bookkeeping. A record holds the field values of one row. The interesting engineering is inside the page, so let us zoom in.

3.2 Inside a page: the slotted-page layout

The naive layout, records packed one after another from the top of the page, has a fatal flaw: delete a record in the middle and either you leave a hole, or you shift everything after it and every record's position changes. Since indexes will point at records by position, moving records would mean updating every index on every delete. The slotted page solves this with one indirection.



slots grow downward from the header; tuples grow upward from the bottom; free space sits between and shrinks from both ends

Mechanism: Slotted page

- A **header** at the top records how many slots are in use and where free space begins and ends.
- A **slot directory** grows downward from the header. Slot i holds the *offset* (and length) of record i within the page.
- **Tuples** grow upward from the bottom of the page.
- **Free space** sits in the middle, shrinking from both ends as records are added.

A record is identified by **(page number, slot number)**, never by its byte offset.

3.3 Why the slot directory is clever

Everything good about this layout follows from the indirection.

- **Stable row ids.** A row's identity is (page, slot). In PostgreSQL this is the `ctid`, and it is what indexes store. The slot holds the tuple's *current* offset, so the tuple can be moved within the page (to reclaim space after deletes, say) without changing its id and without touching any index.
- **Cheap delete.** Deleting a row just marks its slot free; the space is reclaimed later by compacting tuples toward the bottom and updating the slots.
- **Variable-length rows are easy.** Because each slot records both offset and length, records of different sizes coexist without any arithmetic assumptions.

Key insight. One level of indirection (slot → offset) buys stable identifiers and flexible space management at the cost of a 4-byte slot per record. Almost every storage system uses this trick.

3.4 Fixed- versus variable-length records

Trade-offs: Record formats

Fixed-length	**Variable-length**
Every record is the same size.	Records differ in size (e.g. a `text` column).
Field k is at a fixed offset: simple arithmetic, no parsing.	Need per-field offsets or delimiters to find field k.
Fast, but wastes space: a 10-character name in a 100-byte field is 90 bytes of padding.	Compact, but every field access first reads the offsets.
	A **null bitmap** at the front marks absent fields so they take no space at all.

And when a single value is too big for a page, a 2 MB document in a `text` column, say? It is compressed and, if still too large, stored *out of line* in a separate overflow table, with the main record holding only a pointer. In PostgreSQL this mechanism is called **TOAST** (The Oversized-Attribute Storage Technique). The effect is that scanning the main table stays fast even when some columns hold huge values.

Worked example: Pause and check: how many rows per page?

Assume an 8 KB page (8192 bytes) with about 24 bytes of header, fixed-length records of 100 bytes, and a 4-byte slot per record.
1. How many records fit on one page?
2. How many pages to store 1,000,000 records? Roughly how many MB?

1: Each record costs $100 + 4 = 104$ bytes (its data plus its slot). Usable space is $8192 - 24 = 8168$ bytes. $\lfloor 8168/104 \rfloor = 78$ records per page. Note the floor: the 79th record would not fit, and records do not straddle pages.
2: $1{,}000{,}000/78 \approx 12{,}821$ pages. $12{,}821 \times 8$ KB ≈ 100 MB.
The payoff of the calculation: knowing the record size lets you predict a table's size *and* its scan cost (12,821 page reads) before you load a single row. Do this back-of-the-envelope whenever you design a table.

Summary

Table → files → pages → records. Inside a page, the slotted layout keeps a slot directory growing down and tuples growing up; a record's id is (page, slot), which stays valid when the tuple moves. Variable-length records use offsets and a null bitmap; oversized values go out of line (TOAST). Records per page = $\lfloor$usable bytes/(record + slot)$\rfloor$.

4 File Organizations

4.1 Conceptual overview

Given pages full of records, how do we arrange the records across the pages? Three classic answers:

- **Heap (unordered):** a record goes wherever there is room.
- **Sorted:** records are kept in order of a search key.
- **Hashed:** a hash of the key decides which page (bucket) the record lives in.

Each is good at some operations and bad at others. We compare them by the only currency we have: pages touched.

4.2 Heap files

Mechanism: Heap file

- **Insert:** drop the record on any page with room. Roughly 1 page write (plus finding a page with space, which a free-space map makes cheap).
- **Find one row by key, no index:** may have to scan the whole file. B pages worst case, $B/2$ average if present.
- **Full scan:** B pages, sequential.

The heap is the default, general-purpose choice. It is excellent for inserts and for reading everything, and terrible for finding one row on its own. That weakness is exactly what indexes (Chapter 3) fix, which is why real systems use "heap file plus indexes" far more often than sorted or hashed files. PostgreSQL tables are heaps.

4.3 Sorted files

Mechanism: Sorted file

- Records kept in order of a search key across the pages.
- **Equality search:** binary search over pages, about $\log_2 B$ page reads.
- **Range search:** binary search to the start, then read forward; $\log_2 B$ plus the pages holding the matches, which are contiguous.
- **Insert:** costly. Keeping order may mean shifting records across many pages, up to B.

Fast reads by key, slow writes: that is the trade. Pure sorted files are rare in practice, because the B+ tree (Chapter 3) gives most of the read benefit without the insert pain.

4.4 Hashed files

Mechanism: Hashed file

- A hash function maps a key to a page, the **bucket**: page $= h(\text{key}) \bmod N$.
- **Equality lookup:** compute the hash, go straight to the bucket. About 1 page read on average.
- **Range queries:** impossible to do efficiently. Nearby keys hash to unrelated buckets, so "all rows between 100 and 200" means scanning everything.
- Performance degrades with collisions (too many keys in one bucket, overflow pages) and skew.

Excellent for "find the row with this exact key"; useless for "find all rows between X and Y".

4.5 Cost at a glance

Operation	Heap	Sorted	Hashed
Full scan	B	B	B
Equality search	B (avg $B/2$)	$\log_2 B$	~ 1
Range search	B	$\log_2 B$+ matches	B (bad)
Insert	~ 1	up to B	~ 1

Worked example: Putting numbers in: $B = 10{,}000$ pages

Operation	Heap	Sorted	Hashed
Find `id = 4275`	5,000 avg	14	1
All ids in 4000..4300 (say 5 pages of matches)	10,000	$14 + 5 = 19$	10,000
Insert one row	1	up to 10,000	1
Full scan	10,000	10,000	10,000

$\log_2 10{,}000 \approx 13.3$, rounded up to 14. Look at the range row: sorted wins by a factor of 500, and hashed is no better than heap. Look at the insert row: sorted can be 10,000 times worse. There is no single winner; the right organisation depends on the mix of operations in the workload.

Trade-offs: Reading the table

No organisation wins everywhere. Heap favours inserts and scans; sorted favours reads by key, especially ranges; hashed favours exact-match lookups. The practical resolution, and the subject of Chapter 3, is to store the data as a heap and add *indexes*, which are separate structures that give sorted-file or hashed-file lookup speed on top of the heap without paying the heap's price for inserts.

Summary

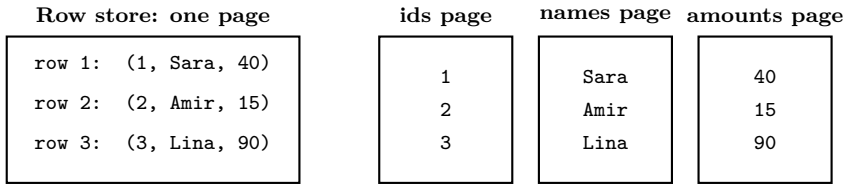
Heap: cheap insert, B to search. Sorted: $\log_2 B$ to search and efficient ranges, but expensive inserts. Hashed: about 1 page for equality, useless for ranges. Real systems use a heap plus indexes.

5 Row Versus Column Storage

5.1 Two ways to place a row's fields

Chapter 1 introduced row and column stores at the logical level. Now we can see them at the page level, which is where the performance difference actually comes from.

- **Row store (NSM, N-ary Storage Model):** all fields of a row are stored together, and a page holds whole rows.
- **Column store (DSM, Decomposition Storage Model):** all values of one column are stored together, and a page holds a slice of one column.



5.2 Why columns win for analytics, and rows for transactions

Consider `SELECT AVG(amount) FROM orders` on a table with 20 columns.

- In a row store, every page holds whole rows, so to see the `amount` of every row you must read *every page of the table*, dragging the other 19 columns through memory for nothing.
- In a column store, the `amount` values are on their own pages. You read only those: roughly one twentieth of the data.
- Furthermore, a column's values all share one type and are often similar (sorted ids, repeated country codes), so they compress extremely well: run-length encoding, dictionary encoding, delta encoding. Fewer bytes on disk means fewer pages to read.

Less data read *times* better compression is why analytical engines are column stores.

And why rows win for transactions: "update customer 42's address" means reading and writing one record. In a row store that is one page. In a column store the record's fields are scattered over 20 pages, one per column, so a single-row update becomes 20 page reads and 20 page writes. OLTP does single-record work all day.

Worked example: Pause and check: estimate the saving

A table has 20 columns and 1,000,000 rows. A query sums one column. Ignoring headers and compression, roughly how much less data does a column store read than a row store? Why might the real saving be larger?

The row store must read pages containing all 20 columns' worth of bytes; the column store reads 1 of 20. So about 20× less I/O. The real saving is usually bigger, because a single-type column compresses much better than mixed-type rows, so the column occupies fewer pages than one twentieth of the table.

5.3 Matching layout to workload

	Row store	Column store
Best for	OLTP	OLAP
Typical access	whole records	few columns, many rows
Writes	fast single-row	better in bulk
Compression	modest	excellent
Examples	PostgreSQL heap, MySQL InnoDB	DuckDB, Parquet files, data warehouses

A middle ground exists too: **PAX** and other hybrid layouts store column groups *within* a page, so a page still holds a set of whole rows (good for OLTP-style fetches) but arranges them column-wise inside (good for cache behaviour and compression). Chapter 12 returns to this in the lakehouse setting.

Summary

Row store: a page holds whole rows; one record is one page; best for OLTP. Column store: a page holds one column; scanning one column reads a fraction of the data and compresses well; best for OLAP. A 20-column table gives roughly a 20× I/O saving on a single-column aggregate, more with compression.

6 How PostgreSQL Stores a Table

6.1 The layout in practice

Everything above is not abstract; you can see all of it in PostgreSQL.

- Each table lives in one or more files (segments of up to 1 GB each) in the data directory, named by an internal id.
- Pages are 8 KB. Rows are stored in a heap, in slotted pages, as a row store.
- Each tuple has a header (transaction ids, flags), then a null bitmap, then the field data.
- Big attributes are pushed aside by TOAST.

- Two helpers sit beside the table: a **free-space map** (which pages have room, for fast inserts) and a **visibility map** (which pages have only rows visible to every transaction, used by scans and vacuum).

Interface / API: Seeing the storage yourself

```
-- physical size of a table, and how many 8 KB pages that is
SELECT pg_size_pretty(pg_relation_size('orders'));
SELECT pg_relation_size('orders') / 8192 AS pages;

-- the page size
SHOW block_size;                          -- 8192

-- every row's physical address: (page, slot)
SELECT ctid, id, amount FROM orders LIMIT 5;
--  ctid  | id | amount
--  (0,1) | 10 |   40      <- page 0, slot 1
--  (0,2) | 11 |   15
--  (0,3) | 12 |   90

-- peek inside page 0 (needs the pageinspect extension)
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT lp, lp_off, lp_len, t_xmin, t_xmax
FROM   heap_page_items(get_raw_page('orders', 0));
--  lp | lp_off | lp_len | t_xmin | t_xmax
--   1 |   8152 |     40 |    731 |      0     <- slot 1 ->
    offset 8152, 40 bytes
--   2 |   8112 |     40 |    731 |      0
--   3 |   8072 |     40 |    731 |      0
```

Read the `heap_page_items` output against the slotted-page diagram. `lp` is the slot number; `lp_off` is the byte offset of the tuple within the page; `lp_len` is its length. Notice the offsets *decrease* as the slot number increases (8152, 8112, 8072): tuples really are growing upward from the bottom of the page, exactly as drawn. The `ctid` you saw in the previous query is the (page, slot) pair from this table, made visible. `t_xmin` and `t_xmax` are the transaction ids that created and deleted the tuple; they belong to Chapter 7.

Technique: A five-minute experiment worth doing

```
CREATE TABLE t (id int, payload char(100));
INSERT INTO t SELECT g, 'x' FROM generate_series(1, 1000000) g;
SELECT pg_relation_size('t') / 8192 AS pages;          --
    compare with your estimate
SELECT count(*) FROM heap_page_items(get_raw_page('t', 0));  --
    rows on page 0
DELETE FROM t WHERE id = 5;
SELECT lp, lp_off, lp_len FROM heap_page_items(get_raw_page('t',
    0)) WHERE lp = 5;
```

Predict the page count with the formula from Section 3 before running it, then explain any difference (hint: PostgreSQL's tuple header is about 24

bytes, and it aligns fields to 8-byte boundaries). After the delete, the slot is still there but the tuple is marked dead; `VACUUM t` reclaims it.

6.2 Why the DBMS manages its own pages instead of trusting the OS

The OS already caches file pages in memory. So why does a DBMS bother to manage its own buffer pool? Because the DBMS knows things the OS does not:

- It knows the **access pattern**. A big sequential scan should not evict every useful page from the cache; the OS's generic least-recently-used policy would do exactly that. The DBMS can say "these pages are a one-pass scan, drop them first".
- It must control **write ordering** for durability. The log record describing a change must reach disk *before* the data page does (write-ahead logging, Chapter 9). The OS decides on its own when to write dirty pages, so the DBMS cannot leave that to it.
- It can **prefetch** intelligently (it knows the next page of the scan) and avoid **double-buffering** the same page once in the OS cache and once in its own.

So it keeps its own buffer pool, the subject of Chapter 6.

7 Tricks, Techniques and Classic Problems

Debugging: Tools for storage questions

- `pg_relation_size('t')`, `pg_total_relation_size('t')` (includes indexes and TOAST), `pg_size_pretty(...)`.
- `SELECT relpages, reltuples FROM pg_class WHERE relname = 't';`: the page count and row estimate the optimiser uses.
- `EXPLAIN (ANALYZE, BUFFERS) <query>`: shows pages read from the buffer pool ("shared hit") versus from disk ("read"). This is the cost model made visible.
- `pageinspect`: `get_raw_page`, `heap_page_items`, `page_header`.
- `VACUUM VERBOSE t`: how many dead tuples were reclaimed and how many pages were compacted.
- Operating-system level: `iostat -x 1` to watch reads per second and whether the disk is the bottleneck.

Technique: Back-of-the-envelope storage estimation

1. Estimate the record size: sum the column sizes, add the tuple header (about 24 bytes in PostgreSQL), round up to 8-byte alignment.
2. Add the slot (4 bytes). Records per page $= \lfloor (8192 - 24)/(\text{record} + 4) \rfloor$.
3. Pages $= \lceil \text{rows/records per page} \rceil$. Size $=$ pages $\times$ 8 KB.
4. Scan cost $=$ pages. Point lookup without index $\approx$ pages/2. With a B+

tree (Chapter 3), about 3–4.
5. Compare with `pg_relation_size` after loading. A large gap usually means dead tuples (run `VACUUM`) or TOAST.

Warning / Gotcha: Common student mistakes

- **Counting bytes instead of pages.** Reading one 50-byte row costs a full 8 KB page. Two rows on the same page cost one read, not two.
- **Forgetting the floor.** 78.5 records per page means 78. Records do not straddle pages.
- **Forgetting the slot.** Each record costs its size *plus* 4 bytes of slot directory.
- **"Absent key costs $B/2$."** No: if the key is absent, a heap scan must read all B pages to be sure. $B/2$ is the average for a *present, unique* key.
- **Thinking a hashed file helps range queries.** It does not; adjacent keys land in unrelated buckets.
- **Assuming column stores are simply better.** They are worse for single-row reads and writes, by a factor of the number of columns.
- **Believing the second run's timing.** A fast second run may just mean the pages are now cached. Measure with a cold cache or with `BUFFERS` to see hits versus reads.

Worked example: Classic exam and interview problems

P1. A heap file has $B = 10{,}000$ pages and no index. Cost to scan? To find a unique key, worst and average? To find a key that is not there?
Scan: 10,000. Worst case: 10,000. Average if present: 5,000. Absent: 10,000, always.
P2. 8 KB pages, 24-byte header, 100-byte fixed records, 4-byte slots. Records per page, and pages for one million records?
$\lfloor 8168/104 \rfloor = 78$ per page; $\lceil 10^6/78 \rceil = 12{,}821$ pages, about 100 MB.
P3. Same table, but records are 200 bytes. What changes?
$\lfloor 8168/204 \rfloor = 40$ per page; 25,000 pages, about 195 MB. Doubling the record size roughly doubles the scan cost. This is why "just add a column" is never free.
P4. Why does a slotted page store an offset in the slot instead of the tuple itself?
So the tuple can move within the page (during compaction after deletes) without changing its (page, slot) id, which indexes rely on. Also to support variable-length tuples.
P5. 20 columns, one million rows, sum one column. Row store versus column store I/O?
About 20× less for the column store before compression; more after, because same-type columns compress well.
P6. Which file organisation for: (a) a lookup table by exact

product code; (b) "all orders between two dates"; (c) an append-only event log?
(a) hashed (or a hash index on a heap). (b) sorted by date (or a B+ tree on date). (c) heap: inserts are cheap and reads are full scans anyway.

P7. Why does an HDD make random reads so much worse than sequential ones, and what does a DBMS do about it?
Every random read moves the arm (a seek of about 10 ms); sequential reads keep it still and stream at full bandwidth. The DBMS lays related rows out contiguously, reads in large runs, and prefetches, turning random patterns into sequential ones.

P8. Name two things the DBMS knows that the OS page cache does not.
The access pattern (a one-pass scan should not evict hot pages) and the required write ordering (log before data). Hence its own buffer pool.

Summary: Chapter 2 in six lines

1. Memory is a hierarchy; disk is 10^4–$10^5\times$ slower than RAM, so we count pages read, not instructions.
2. Data moves in fixed-size pages (8 KB in PostgreSQL); you pay per page, and sequential beats random.
3. The slotted page gives stable (page, slot) row ids and flexible space use; records per page $= \lfloor \text{usable}/(\text{record} + \text{slot}) \rfloor$.
4. Heap favours inserts and scans; sorted favours key and range reads; hashed favours equality. Real systems use heap plus indexes.
5. Row storage suits OLTP; column storage suits OLAP and compresses well; roughly a $20\times$ saving on one column of twenty.
6. PostgreSQL exposes all of this: `ctid`, `pg_relation_size`, `pageinspect`. Next: indexing, finding a row in $\log B$ instead of B.